The Reader's Engine: Stage Two of a Verified Reader

Working paper v0.1, Project secret3, August 11, 2026

Abstract

Stage one of the C5 campaign gave closed programs a quoted alphabet. This stage gives them a verified engine. A total function simStep computes the leftmost one-step reduction of any coded program, and three compiled theorems pin its meaning to the calculus exactly. Soundness, simSound: whenever the engine reports a step, the embedded term really takes that step, position and rule included. Completeness of silence, simNormal: whenever the engine reports no step, the embedded term has no step at all, at any position, by any rule; silence means normal form, proved as a genuine negation, quantified over every candidate successor. And fueled soundness, simRunSteps: every n-step run of the engine is a real walk of the embedded term, assembled step by step from simSound. Together these make the engine a certified reference semantics: the reader's behavior is now a specification with theorems, and stage three's task is reduced to building a term inside the calculus that meets it. The paper also reports the proof-engineering findings, two of them general: case expressions must not appear in type signatures, where distinct elaborations refuse to unify, so decision structure belongs in named helper functions; and Idris2' s totality checker distrusts recursion under with-abstraction, so the working pattern is to package each case analysis as a non-recursive view lemma returning evidence, leaving every recursive function with-free and plainly structural.

1. What stage two is, and why it comes before the internal term

A self-interpreter needs three things: a representation (stage one), a semantics for what reading means (this stage), and a term inside the calculus that implements it (stage three). Building the term first would be building to no specification: the correctness statement of the future internal reader is exactly agreement with this engine, on the alphabet, under embed. So the engine is not the fallback; it is the specification, delivered with full proofs, and the fallback position named in the earlier papers is hereby upgraded: whatever happens to stage three, the project now possesses a verified evaluator for its calculus at the code level, which is a standing result on its own.

2. The engine

```
simStep : Code -> Maybe Code
```

computes leftmost reduction by a complete, non-overlapping spine analysis: atoms are silent; the two redex shapes, a K with two arguments and an S with three, fire at the top; every other application descends left first, then right, through a named helper descR that packages the right descent. Non-overlap matters twice: it makes the function's equations usable as rewrite rules in proofs, and it makes the leftmost strategy visible in the clause structure rather than buried in a match order. Fueled iteration, simRun, runs the engine n steps or to silence, whichever comes first.

3. The three theorems

```
simSound    : simStep c = Just c' -> Step  (embed c) (embed c')
simNormal   : simStep c = Nothing ->
              (u : Term) -> Step (embed c) u -> Void
simRunSteps : Steps (embed c) (embed (simRun n c))
```

Soundness is structural induction with each successful clause producing its step: the two redexes give RedK and RedS on the nose, the descents give CtxL and CtxR around the recursive call. Completeness of silence is the stronger and more laborious direction: for each silent shape, every one of the four step constructors must be refuted at every position, the atom cases by the uninhabitedness of steps from S and K, the spine cases by unification mismatch against the redex patterns, and the recursive cases by the induction hypotheses on both children. Nothing is asserted, nothing postulated; the negation is constructed. Fueled soundness folds simSound down a run. The three together say: the engine takes exactly the steps the calculus takes, stops exactly where the calculus stops, and its runs are the calculus's runs.

4. Proof engineering, reported

Two findings general enough to record. First, case expressions in type signatures do not unify: two syntactically identical case terms elaborate to distinct case blocks, so a lemma whose statement mentions one can never be applied to a goal mentioning the other. The cure is to name the decision as a top-level function (here descR) and state every lemma against the name. Second, the totality checker distrusts recursion under with: recursive calls inside with-children, and especially inside nested with-blocks, are reported as possibly non-terminating even when plainly structural. The cure that worked uniformly: package each case split as a non-recursive view lemma (descRJust, sViewQSa, sViewDy, and the descRNothing silence-propagator) that performs the with internally and returns a dependent pair of evidence; the recursive functions then consume views and are with-free, so their descent is syntactically structural and the checker is satisfied. simNormal uses the dual arrangement, with-free auxiliary refuters fed by equalities. The module's earlier quantity lesson and this stage's view lesson are of a kind: the checker rewards making information explicit, terms in indices there, decisions as data here.

5. What stage three inherits

The internal reader's correctness statement can now be written before its term exists: a closed term Sim such that for all codes c, if simStep c = Just c' then Sim applied to quote c walks to quote c', and if simStep c = Nothing then Sim applied to quote c walks to a designated normal tag. The alphabet API of stage one supplies the casing, the vanishing engine supplies the construction, and this stage's theorems transfer any such agreement to the calculus itself through simSound and simNormal. The named risk stands: the internal casing over quoted spines is deep, and the correctness induction will be long. The named asset is new: it now has a spec.

Appendix A: Methods

All theorems are in the shared proof module Proofs.idr, 2,074 lines after this paper's additions, %default total, no postulates, no holes; idris2 —check Proofs.idr is the entire verification, and the entry point prints the theorem inventory, including simSound, simNormal, and simRunSteps.

Appendix B: The engine section (module part R)

```
--------------------------------------------------------------------
-- Tier 2, part R: the reader's engine (C5 stage two)
```

```
------------------------------------------------------------------

||| right descent: x is finished, try the argument side
descR : Code -> Maybe Code -> Maybe Code
descR x (Just y') = Just (QApp x y')
descR x Nothing   = Nothing

||| leftmost one-step simulation at the code level
simStep : Code -> Maybe Code
simStep QS = Nothing
simStep QK = Nothing
simStep (QApp QS y) = descR QS (simStep y)
simStep (QApp QK y) = descR QK (simStep y)
simStep (QApp (QApp QK a) y) = Just a
simStep (QApp (QApp QS a) y) =
  case simStep a of
    Just a' => Just (QApp (QApp QS a') y)
    Nothing => descR (QApp QS a) (simStep y)
simStep (QApp (QApp (QApp QS a) b) y) =
  Just (QApp (QApp a y) (QApp b y))
simStep (QApp (QApp (QApp QK a) b) y) = Just (QApp a y)
simStep (QApp (QApp (QApp (QApp w z) a) b) y) =
  case simStep (QApp (QApp (QApp w z) a) b) of
    Just x' => Just (QApp x' y)
    Nothing => descR (QApp (QApp (QApp w z) a) b) (simStep y)

justInj : Just a = Just b -> a = b
justInj Refl = Refl

||| silence propagates: a silent right descent means a silent argument
descRNothing : (x : Code) -> (m : Maybe Code) ->
  descR x m = Nothing -> m = Nothing
descRNothing x (Just _) prf impossible
descRNothing x Nothing  prf = Refl

||| the two atoms take no step
noStepCS : Step CS u -> Void
noStepCS step impossible

noStepCK : Step CK u -> Void
noStepCK step impossible

||| a successful right descent, analyzed
descRJust : (x : Code) -> (m : Maybe Code) -> (c' : Code) ->
  descR x m = Just c' -> (y' : Code ** (m = Just y', c' = QApp x y'))
descRJust x (Just y0) c' prf = (y0 ** (Refl, sym (justInj prf)))
descRJust x Nothing   c' prf impossible

||| view of a successful step on an S-headed spine of two
sViewQSa : (a, y, c' : Code) ->
  simStep (QApp (QApp QS a) y) = Just c' ->
  Either (a' : Code ** (simStep a = Just a', c' = QApp (QApp QS a') y))
         (y' : Code ** ( simStep a = Nothing
                       , simStep y = Just y'
                       , c' = QApp (QApp QS a) y'))
```

```
sViewQSa a y c' prf with (simStep a) proof eqa
  sViewQSa a y c' prf | Just a' =
    Left (a' ** (Refl, sym (justInj prf)))
  sViewQSa a y c' prf | Nothing =
    let (y' ** (eqy, eqc)) = descRJust (QApp QS a) (simStep y) c' prf
    in Right (y' ** (Refl, eqy, eqc))

||| view of a successful step on a deep left spine
sViewDy : (w, z, a, b, y, c' : Code) ->
  simStep (QApp (QApp (QApp (QApp w z) a) b) y) = Just c' ->
  Either (x' : Code ** ( simStep (QApp (QApp (QApp w z) a) b) = Just x'
                       , c' = QApp x' y))
         (y' : Code ** ( simStep (QApp (QApp (QApp w z) a) b) = Nothing
                       , simStep y = Just y'
                       , c' = QApp (QApp (QApp (QApp w z) a) b) y'))
sViewDy w z a b y c' prf
  with (simStep (QApp (QApp (QApp w z) a) b)) proof eqx
  sViewDy w z a b y c' prf | Just x' =
    Left (x' ** (Refl, sym (justInj prf)))
  sViewDy w z a b y c' prf | Nothing =
    let (y' ** (eqy, eqc)) =
          descRJust (QApp (QApp (QApp w z) a) b) (simStep y) c' prf
    in Right (y' ** (Refl, eqy, eqc))

||| soundness: every simulated step is a real step of the embedded term
simSound : (c, c' : Code) -> simStep c = Just c' ->
  Step (embed c) (embed c')
simSound QS c' prf impossible
simSound QK c' prf impossible
simSound (QApp QS y) c' prf =
  case descRJust QS (simStep y) c' prf of
    (y' ** (eqy, eqc)) =>
      rewrite eqc in CtxR (simSound y y' eqy)
simSound (QApp QK y) c' prf =
  case descRJust QK (simStep y) c' prf of
    (y' ** (eqy, eqc)) =>
      rewrite eqc in CtxR (simSound y y' eqy)
simSound (QApp (QApp QK a) y) c' prf =
  rewrite sym (justInj prf) in RedK
simSound (QApp (QApp QS a) y) c' prf =
  case sViewQSa a y c' prf of
    Left (a' ** (eqa, eqc)) =>
      rewrite eqc in CtxL (CtxR (simSound a a' eqa))
    Right (y' ** (_, eqy, eqc)) =>
      rewrite eqc in CtxR (simSound y y' eqy)
simSound (QApp (QApp (QApp QS a) b) y) c' prf =
  rewrite sym (justInj prf) in RedS
simSound (QApp (QApp (QApp QK a) b) y) c' prf =
  rewrite sym (justInj prf) in CtxL RedK
simSound (QApp (QApp (QApp (QApp w z) a) b) y) c' prf =
  case sViewDy w z a b y c' prf of
    Left (x' ** (eqx, eqc)) =>
      rewrite eqc in
      CtxL (simSound (QApp (QApp (QApp w z) a) b) x' eqx)
    Right (y' ** (_, eqy, eqc)) =>
```

```
      rewrite eqc in CtxR (simSound y y' eqy)

mutual
  ||| completeness of silence: when the engine says Nothing,
  ||| the embedded term has no step at all, anywhere
  simNormal : (c : Code) -> simStep c = Nothing ->
    (u : Term) -> Step (embed c) u -> Void
  simNormal QS prf u step = noStepCS step
  simNormal QK prf u step = noStepCK step
  simNormal (QApp QS y) prf _ RedK impossible
  simNormal (QApp QS y) prf _ RedS impossible
  simNormal (QApp QS y) prf _ (CtxL s) = noStepCS s
  simNormal (QApp QS y) prf _ (CtxR s) =
    simNormal y (descRNothing QS (simStep y) prf) _ s
  simNormal (QApp QK y) prf _ RedK impossible
  simNormal (QApp QK y) prf _ RedS impossible
  simNormal (QApp QK y) prf _ (CtxL s) = noStepCK s
  simNormal (QApp QK y) prf _ (CtxR s) =
    simNormal y (descRNothing QK (simStep y) prf) _ s
  simNormal (QApp (QApp QK a) y) prf u step impossible
  simNormal (QApp (QApp QS a) y) prf u step with (simStep a) proof eqa
    simNormal (QApp (QApp QS a) y) prf u step | Just a' impossible
    simNormal (QApp (QApp QS a) y) prf u step | Nothing =
      simNormalSyAux a y eqa
        (descRNothing (QApp QS a) (simStep y) prf) u step
  simNormal (QApp (QApp (QApp QS a) b) y) prf u step impossible
  simNormal (QApp (QApp (QApp QK a) b) y) prf u step impossible
  simNormal (QApp (QApp (QApp (QApp w z) a) b) y) prf u step
    with (simStep (QApp (QApp (QApp w z) a) b)) proof eqx
    simNormal (QApp (QApp (QApp (QApp w z) a) b) y) prf u step | Just x'
      impossible
    simNormal (QApp (QApp (QApp (QApp w z) a) b) y) prf u step | Nothing =
      simNormalDyAux w z a b y eqx
        (descRNothing (QApp (QApp (QApp w z) a) b) (simStep y) prf) u step

  simNormalSyAux : (a, y : Code) ->
    simStep a = Nothing -> simStep y = Nothing ->
    (u : Term) -> Step (App (App CS (embed a)) (embed y)) u -> Void
  simNormalSyAux a y eqa eqy _ RedK impossible
  simNormalSyAux a y eqa eqy _ RedS impossible
  simNormalSyAux a y eqa eqy _ (CtxL RedK) impossible
  simNormalSyAux a y eqa eqy _ (CtxL RedS) impossible
  simNormalSyAux a y eqa eqy _ (CtxL (CtxL s)) = noStepCS s
  simNormalSyAux a y eqa eqy _ (CtxL (CtxR s)) = simNormal a eqa _ s
  simNormalSyAux a y eqa eqy _ (CtxR s) = simNormal y eqy _ s

  simNormalDyAux : (w, z, a, b, y : Code) ->
    simStep (QApp (QApp (QApp w z) a) b) = Nothing ->
    simStep y = Nothing ->
    (u : Term) ->
    Step (App (embed (QApp (QApp (QApp w z) a) b)) (embed y)) u -> Void
  simNormalDyAux w z a b y eqx eqy _ RedK impossible
  simNormalDyAux w z a b y eqx eqy _ RedS impossible
  simNormalDyAux w z a b y eqx eqy _ (CtxL s) =
    simNormal (QApp (QApp (QApp w z) a) b) eqx _ s
```

```
  simNormalDyAux w z a b y eqx eqy _ (CtxR s) =
    simNormal y eqy _ s

||| fueled runs of the engine
simRun : Nat -> Code -> Code
simRun Z c = c
simRun (S n) c =
  case simStep c of
    Just c' => simRun n c'
    Nothing => c

||| every fueled run is a real walk of the embedded term
simRunSteps : (n : Nat) -> (c : Code) ->
  Steps (embed c) (embed (simRun n c))
simRunSteps Z c = Done
simRunSteps (S n) c with (simStep c) proof eq
  simRunSteps (S n) c | Just c' =
    Then (simSound c c' eq) (simRunSteps n c')
  simRunSteps (S n) c | Nothing = Done
```

References

1. Project secret3, ‘A Quoted Alphabet: Stage One of a Verified Reader’, working paper v0.1, August 11, 2026.
2. Project secret3, ‘A Verified Coin: Type Preservation for the Two-Move Hilbert System’, working paper v0.1, August 11, 2026, section 4, for the quantity lesson this stage’s view lesson parallels.
3. T. Mogensen, ‘Efficient self-interpretation in lambda calculus’, Journal of Functional Programming 2 (1992), 345-364.
4. Project secret3, research agenda v0.8, line C5, August 11, 2026.